코디세이 Pre-course 미션2 해결 가이드

Git과 함께하는 Python 첫 발자국

Python 콘솔 퀴즈 프로그램

미션 요구사항 분석 · 프로그램 설계 · AI 활용 개발 · Git 변경 이력 관리

목차

1. 미션 이해와 개발 방법

- 미션 요구사항 분석 및 생성형 AI 활용 방법

2. Python 퀴즈 프로그램 분석 및 설계

- 프로그램의 전체 실행 흐름 설계

3. 기능 요구사항별 구현 방법

- 요구사항별 해결 방법과 Python 핵심 문법

4. 프로그램 검증 및 Git 버전 관리

- 미션 최종 산출물 확인

01 미션 이해와 개발 방법

■ 생성형 AI를 활용한 미션 해결 과정

- 생성형 AI를 활용하여 미션 요구사항을 분석하고, 프로그램 설계부터 구현·검증까지 단계별로 수행

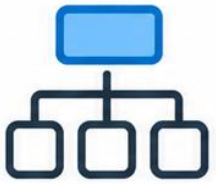
01 요구사항 분석



AI 활용

- 요구사항 분류 및 정리
- 필수 조건 분석
- 구현 우선순위 제안
- 추가 고려사항 확인

02 프로그램 설계



AI 활용

- 실행 흐름 및 구조 설계 제안
- 클래스와 메서드 구조 제안
- 데이터 구조(JSON) 설계 지원
- 파일 구성 및 설계 내용 검토

03 개발 환경 구성



AI 활용

- GitHub 저장소 생성 방법 안내
- README.md 초안 작성 지원
- 프로젝트 파일 구조 제안
- .gitignore 설정 방법 안내

04 기능별 구현



AI 활용

- 필요한 Python 문법 설명
- 요구사항별 코드 생성 지원
- 기능 구현 방법 제안
- 코드 개선 및 리팩토링 조언

05 실행 및 검증



AI 활용

- 오류 원인 분석 및 해결 방법 제안
- 예외 처리 방법 안내
- 실행 결과 확인 및 수정 방향 제안
- 데이터 저장·복원 검증 방법 안내

06 최종 결과물 관리



AI 활용

- 필수 완료 조건 충족 여부 점검
- README.md 최종 수정 지원
- 문서 내용 검토 및 개선 제안
- GitHub 제출 전 최종 확인

01 미션 이해와 개발 방법

■ 미션 필수 요구사항

- Python 객체지향 프로그래밍을 활용하여 퀴즈 게임의 기본 구조와 주요 기능을 구현하고,
  입력값 검증 및 JSON 데이터 관리 기능을 적용하며 Git·GitHub를 통한 버전 관리와 협업 과정을 수행

구분	요구사항 번호	주요 내용
개발 환경 및 버전 관리	01, 13	GitHub 저장소를 생성하고 Commit·Push 및 Clone·Pull을 통해 버전 관리와 동기화 과정을 검증
프로그램 기본 구조	04, 05, 10	Quiz·QuizGame 클래스를 설계하고 초기 퀴즈 5개를 구성하여 객체지향 프로그램의 기본 구조 구현
퀴즈 게임 기능	02, 06, 07, 08, 09	메뉴 실행, 퀴즈 풀이·추가, 목록 조회 및 최고 점수 관리 기능 구현
입력 및 데이터 관리	03, 11	입력값 검증과 예외 처리를 적용하고 JSON 파일을 통해 퀴즈 데이터와 최고 점수를 저장·관리
프로젝트 문서화	12	README.md에 프로젝트 개요, 실행 방법, 주요 기능 및 파일 구조 작성

01 미션 이해와 개발 방법

■ 최종 산출물

- 퀴즈 게임 실행 파일(main.py), 퀴즈 데이터 저장 파일(state.json), 프로젝트 설명 문서(README.md)를 완성하고, GitHub 저장소에 최종 코드와 개발 이력을 반영

산출물	주요 내용
main.py	퀴즈 게임의 주요 기능과 Quiz·QuizGame 클래스를 구현한 Python 실행 파일
state.json	퀴즈 목록과 최고 점수를 JSON 형식으로 저장하는 데이터 파일
README.md	프로젝트 개요, 실행 방법, 주요 기능 및 파일 구조를 설명하는 문서
GitHub 저장소	최종 소스코드와 문서를 관리하고 Commit 10회 이상, 브랜치 생성·Merge 및 Clone·Pull 수행 이력을 확인할 수 있는 저장소

01 미션 이해와 개발 방법

■ 미션 요구사항 기반 단계별 개발 전략

- 미션 요구사항을 분석하여 기본 구조 구축, 핵심 기능 개발, 프로그램 통합 및 최종 검증, Git 협업 검증의 4단계로 개발 과정을 구성

1단계.
기본 구조 설계 및 초기 환경 구축



GitHub 저장소 생성, Quiz·QuizGame 클래스 설계,
초기 퀴즈 5개 및 JSON 데이터 구조 구성
요구사항 01, 04, 05, 10, 11(기초)

main

2단계.
브랜치 생성 및 퀴즈 프로그램 핵심 기능 개발



main에서 feature/quiz-play 브랜치를 생성하고,
메뉴 반복 실행, 입력 검증, 퀴즈 풀이·추가, 목록 조회,
최고 점수 관리 및 데이터 저장
요구사항 02, 03, 06, 07, 08, 09, 11(완성)

feature/quiz-play

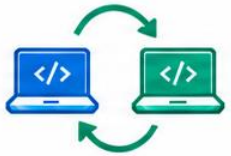
3단계.
브랜치 병합 및 최종 정리



개발 기능별 검증과 Commit·Push 완료 후
main에 Merge하고, 통합 실행 테스트 및 README 최종 작성
요구사항 06(브랜치 병합), 12 및 전체 기능 검증

Merge → main

4단계.
Git 협업 흐름 검증



다른 폴더에 저장소를 Clone하고 변경사항을
Commit·Push한 후, 원래 폴더에서 Pull하여 동기화 확인
요구사항 13

Clone · Push · Pull

01 미션 이해와 개발 방법

■ 생성형 AI를 활용한 미션 해결 시 고려사항

- 전체 프로그램을 한 번에 생성하기보다 요구사항 단위로 분석·설계·구현
- 기능 구현 후 실행 결과를 확인하고, 미션에 지정된 시점에 Commit·Push 수행
- Branch·Merge 및 Clone·Pull은 해당 기능 요구사항에 따라 수행
- 최종 구현 결과와 README.md의 내용이 일치하는지 확인

■ 미션 시작을 위한 초기 프롬프트 예시

- 미션 문서를 생성형 AI에 첨부하고, 요구사항 분석과 단계별 개발을 위한 초기 프롬프트 입력 (아래 예시)

> 첨부한 미션 문서를 분석하여 미션의 목표와 필수 요구사항, 최종 산출물을 이해하기 쉽게 정리해 주세요.
> 미션의 모든 요구사항을 충족할 수 있도록 프로젝트 구조, 프로그램의 주요 기능과 실행 흐름,
> Git 활용 계획을 단계별로 제안해 주세요.
> 아직 코드를 작성하지 말고, 분석 및 설계 결과를 먼저 제시해 주세요. 검토 후 요구사항별로 하나씩 구현하겠습니다.

02 Python 퀴즈 프로그램 분석 및 설계

■ 프로젝트 구조 및 파일 구성 설계

- 생성형 AI가 제안한 프로젝트 구조를 확인하고, 각 폴더와 파일의 역할 및 요구사항과의 관계를 이해

```
codyssey-python-quiz/        # 프로젝트 최상위 폴더
 ├──── .gitignore            # Git 관리 제외 대상 설정
 ├──── README.md             # 프로젝트 설명 및 실행 안내
 ├──── state.json            # 퀴즈 데이터와 최고 점수 저장
 └──── main.py               # Python 퀴즈 게임 실행 파일
```

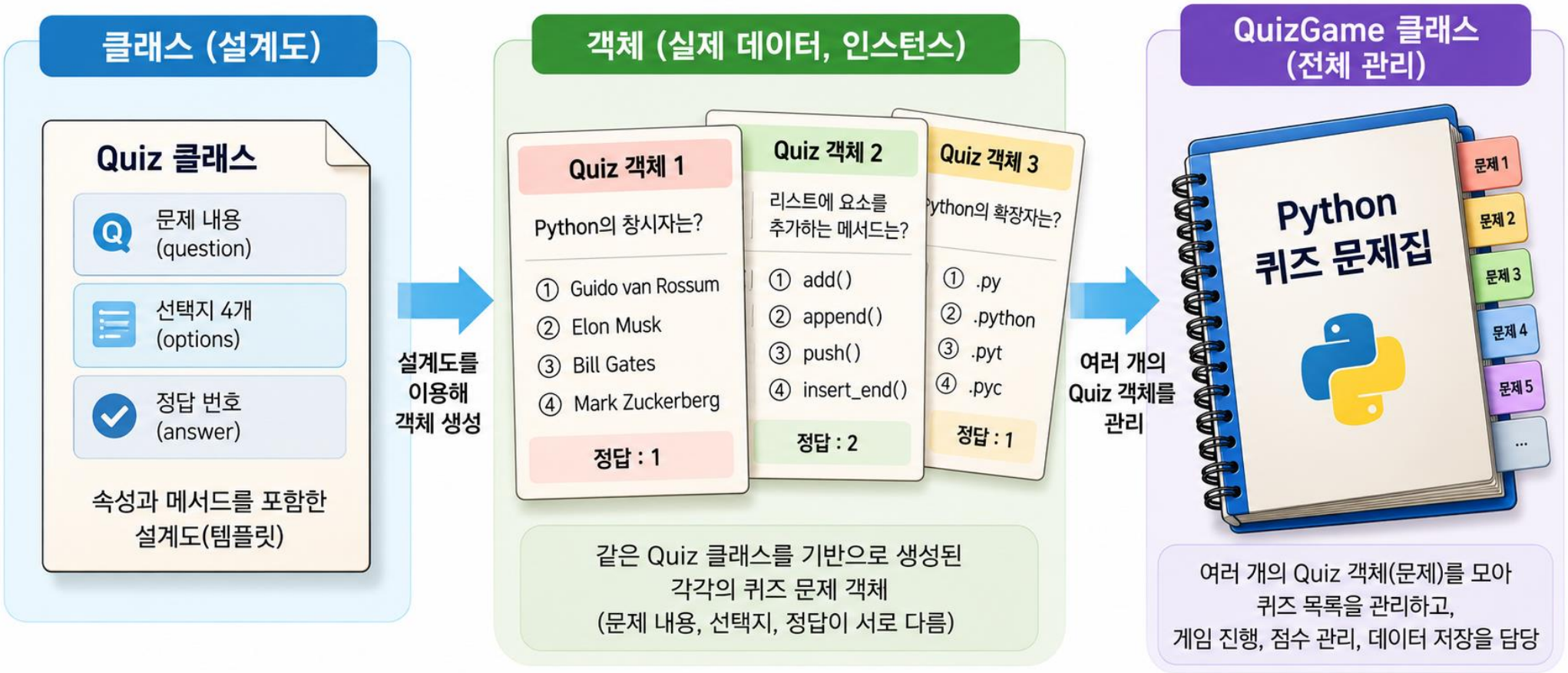
- 프로젝트 폴터 및 파일별 역할

폴더 및 파일	역할
codyssey-python-quiz/	Python 퀴즈 게임 개발에 필요한 파일을 모아 관리하는 프로젝트 최상위 폴더
.gitignore	Git에서 추적하지 않을 파일과 폴더를 지정하는 설정 파일
README.md	프로젝트 개요, 주요 기능, 실행 방법 등을 정리하는 설명 문서
state.json	퀴즈 목록과 최고 점수를 저장하여 프로그램 재실행 시에도 데이터를 유지하는 파일
main.py	Quiz·QuizGame 클래스와 메뉴 및 주요 기능을 구현하는 Python 실행 파일

02 Python 퀴즈 프로그램 분석 및 설계

■ Quiz·QuizGame 클래스 및 초기 데이터 설계

- 객체지향 프로그래밍(OOP, Object-Oriented Programming)은 데이터와 기능을 하나의 객체로 구성하고, 객체 간의 상호작용을 통해 프로그램을 개발하는 방식



02 Python 퀴즈 프로그램 분석 및 설계

■ Quiz·QuizGame 클래스와 Python 코드 구조

Quiz 클래스 (설계도)

```
class Quiz:
  def __init__(self, question, options, answer):
      self.question = question
      self.options = options
      self.answer = answer

  def to_dict(self):
      return {
          "question": self.question,
          "options": self.options,
          "answer": self.answer
      }
```

주요 속성

question	: 문제 내용
options	: 선택지 4개 (리스트)
answer	: 정답 번호

주요 메서드

__init__()	: 객체 초기화
to_dict()	: 딕셔너리로 변환 (저장용)

설계도를 이용해 객체 생성

Quiz 객체 (실제 데이터, 인스턴스)

Quiz 객체 1

```
question = "Python의 창시자는?"
options  = ["Guido van Rossum",
            "Elon Musk",
            "Bill Gates",
            "Mark Zuckerberg"]
answer = 1
```

Quiz 객체 2

```
question = "리스트에 요소를 추가하는 메서드는?"
options  = ["add()", "append()",
            "push()", "insert_end()"]
answer = 2
```

Quiz 객체 3

```
question = "Python의 확장자는?"
options  = [".py", ".python", ".pyt", ".pyc"]
answer = 1
```

여러 개의 Quiz 객체를 관리

QuizGame 클래스 (전체 관리)

```
class QuizGame:
  def __init__(self):
      self.quizzes = []        # 여러 Quiz 객체를 저장
      self.high_score = 0      # 최고 점수
      self.file_path = "state.json"  # 데이터 파일 경로

  def load_data(self):         # 데이터 읽기
      ...

  def save_data(self):         # 데이터 저장
      ...

  def add_quiz(self, quiz):    # 퀴즈 추가
      self.quizzes.append(quiz)

  def play_quiz(self):         # 퀴즈 게임 진행
      ...

  def show_list(self):         # 퀴즈 목록 보기
      ...

  def show_score(self):        # 최고 점수 확인
      ...
```

load_data()
저장된 데이터를 읽어 Quiz 객체로 생성

state_json

```
{
  "quizzes": [
    {
      "question": "Python의 창시자는?",
      "options": ["Guido van Rossum", "Elon Musk", "Bill Gates", "Mark Zuckerberg"],
      "answer": 1
    },
  : {"question":"리스트에 요소를 추가하는 메서드는?", "options":["add()", "append()", "push()", "insert_end()": 2},
    {"question": "Python의 확장자는?", "options": [".py", ".python", ".pyt", ".pyc"], "answer": 1}
  ],
  "high_score": 3
}
```

save_data()
현재 퀴즈 목록과 최고 점수를 저장

02 Python 퀴즈 프로그램 분석 및 설계

■ 초기 데이터 및 JSON 저장 구조 설계

- JSON(JavaScript Object Notation)은 데이터를 키(Key)와 값(Value)으로 표현하는 경량 데이터 교환 형식

개념	설명	퀴즈 프로그램 적용
리스트(List)	여러 데이터를 순서대로 관리하는 자료구조	선택지 4개, 전체 퀴즈 목록
딕셔너리(Dictionary)	키(Key)와 값(Value)의 쌍으로 데이터를 관리하는 Python 자료구조	문제 내용, 선택지, 정답 번호
JSON	구조화된 데이터를 텍스트 형식으로 저장·교환	state.json
데이터 영속성(Persistence)	프로그램 종료 후에도 데이터를 유지하는 특성	퀴즈 목록 및 최고 점수 저장

state.json 예시

```
{
  "high_score": 0,
  "quizzes": [
    {
      "question": "Python의 창시자는?",
      "options": ["Guido van Rossum", "Elon Musk", "Bill Gates", "Mark Zucker
berg"],
      "answer": 1
    }
  ]
}
```

미션 요구사항 05(초기 퀴즈 데이터) 11(JSON 파일 저장 및 불러오기) 생성형 AI가 제안한 JSON 구조와 데이터 저장·불러오기 방식 확인 하기

02 Python 퀴즈 프로그램 분석 및 설계

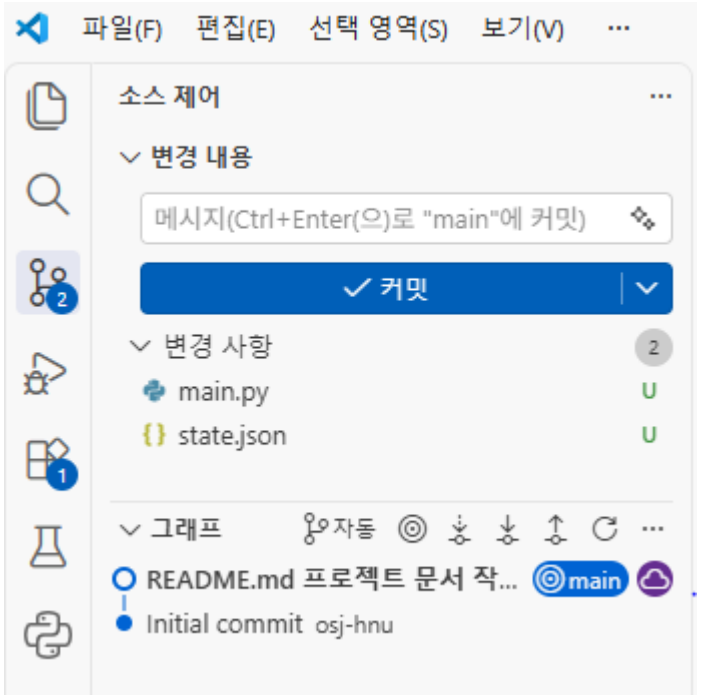
■ Python 프로그램 실행 및 초기 데이터 검증

- 요구사항 04(Quiz 클래스), 05(초기 퀴즈 데이터), 10(QuizGame 클래스), 11(JSON 저장 기능)의 구현 결과를 확인하고 GitHub에 반영(Git 필수 요구사항(01) 및 전체 Commit 10회 이상 조건 반영)

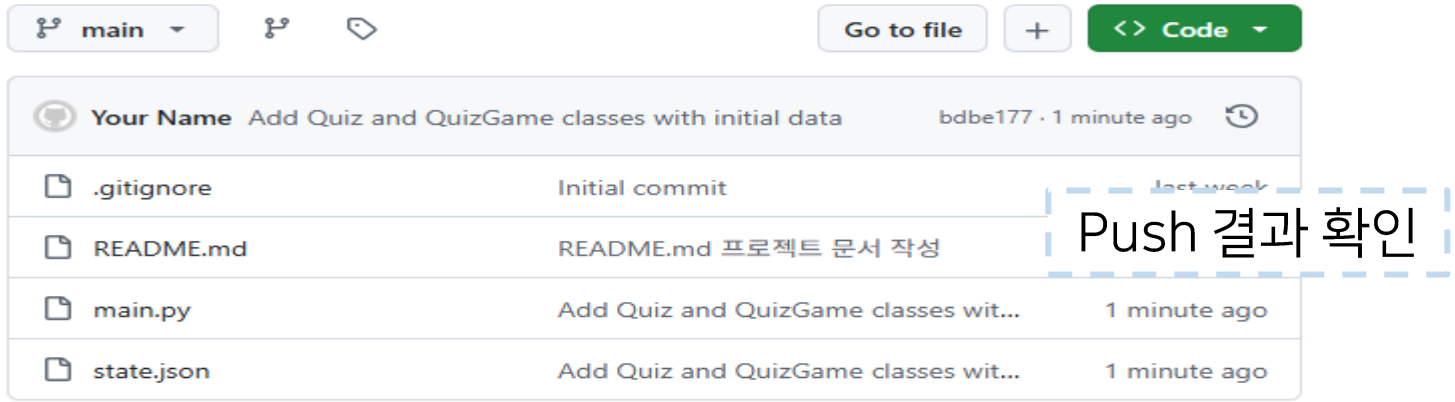
1. 프로그램 실행 - 프로젝트 폴더에서 터미널을 열고 다음 명령어 실행

```
python main.py
```

2. Git Commit, Push – 소스제어 메뉴 또는 터미널 명령어 사용하기 ⇒ gitHub 레파지토리 확인하기




```
git add main.py state.json
git commit -m "Add Quiz and QuizGame classes with initial data"
git push origin main
```



03 기능 요구사항별 구현 방법

■ Git 브랜치 생성 및 퀴즈 프로그램 기능 개발

- Git 브랜치(Branch)는 기존 코드의 상태를 유지하면서 새로운 기능을 독립적으로 개발할 수 있도록 분리한 작업 공간
- main 브랜치에는 실행이 확인된 기본 프로그램을 유지하고, feature/quiz-play 브랜치에서 퀴즈 게임의 주요 기능을 구현
- 기능 개발과 테스트가 완료되면 변경사항을 Commit·Push하고, 최종적으로 main 브랜치에 Merge하여 통합

■ 브랜치 기반 개발 흐름



■ VS Code에서 브랜치 생성

```
git switch main
git pull origin main
git switch -c feature/quiz-play
git push -u origin feature/quiz-play
```

명령어 설명
git switch main 현재 작업 브랜치를 기본 브랜치인 main으로 전환
git pull origin main GitHub의 최신 main 브랜치 변경사항을 로컬 main에 반영
git switch -c feature/quiz-play 현재 main을 기준으로 새로운 기능 개발 브랜치를 생성하고 해당 브랜치로 전환
git push -u origin feature/quiz-play 생성한 브랜치를 GitHub에 등록하고 로컬 브랜치와 원격 브랜치의 추적 관계 설정

03 기능 요구사항별 구현 방법

■ 기능 구현 작업 진행

- GitHub 저장소의 브랜치 선택 메뉴에서 main과 feature/quiz-play가 모두 생성되었는지 확인
- 브랜치 상태를 확인한 후 feature/quiz-play에서 메뉴 구성, 퀴즈 풀이, 퀴즈 추가 등 주요 기능을 단계적으로 구현

■ 기능 요구사항별 구현 순서

순서	구현 기능	학습할 Python 개념	검증 내용
1	메인 메뉴 구현	while, if/elif, input()	5개 메뉴 출력 및 종료
2	입력값 검증	조건문, try/except	잘못된 입력과 예외 처리
3	퀴즈 목록 조회	for, 리스트, 객체 속성	저장된 퀴즈 5개 출력
4	퀴즈 풀이	메서드, 반복문, 조건문	정답 판정 및 점수 계산
5	퀴즈 추가	input(), 객체 생성	새 퀴즈 추가 및 JSON 저장
6	최고 점수 관리	조건문, JSON 읽기·쓰기	최고 점수 갱신 및 유지

- 각 기능은 구현 → 실행 → 검증 → Commit·Push 순서로 진행

03 기능 요구사항별 구현 방법

■ 기능 구현 작업 예시: 메인 메뉴 구현

- 우선 QuizGame 클래스의 run() 메서드를 수정하여 다음 5개 메뉴가 반복 출력되도록 구현

Python 터미널 실행 화면 예시
===== Python 퀴즈 게임 =====
1. 퀴즈 풀기
2. 퀴즈 추가
3. 퀴즈 목록 보기
4. 최고 점수 보기
5. 종료

메뉴를 선택하세요:

VS Code에서 Codex에 입력할 프롬프트예시
현재 main.py의 Quiz와 QuizGame 클래스 구조를 유지하면서 미션 요구사항 02의 메인 메뉴 기능을 구현해 주세요.

1. QuizGame 클래스의 run() 메서드에 5개 메뉴를 출력해 주세요.
2. while 반복문을 사용하여 종료를 선택하기 전까지 메뉴가 반복되도록 해 주세요.
3. if/elif 조건문으로 메뉴 선택을 구분해 주세요.
4. 5번을 선택하면 프로그램을 종료해 주세요.
5. 1~4번 기능은 아직 구현하지 않고, 선택한 기능의 이름만 출력해 주세요.

기존 JSON 저장 및 불러오기 기능은 변경하지 마세요.
수정한 코드와 사용된 Python 문법을 설명해 주세요.

- 각 기능은 구현 → 실행 → 검증 → Commit·Push 순서로 진행

```
[ Python 퀴즈 게임 ]
1. 퀴즈 풀기
2. 퀴즈 추가
3. 퀴즈 목록 보기
4. 최고 점수 확인
5. 종료
원하는 메뉴 번호를 입력하세요: 1

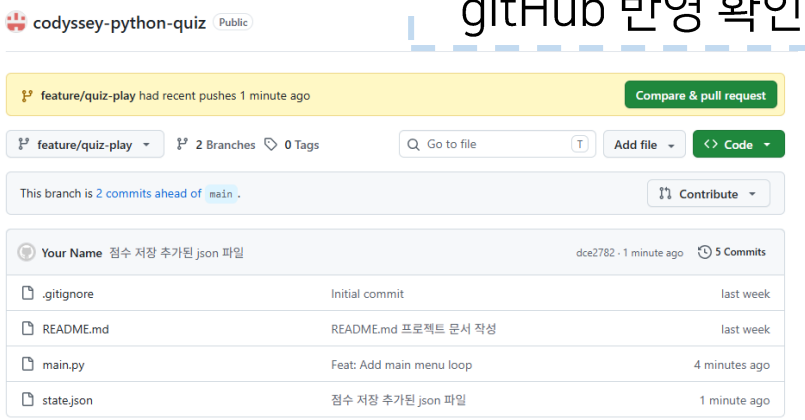
🚀 퀴즈를 시작합니다! (총 5문제)
```

코드 실행

```
git add main.py state.json
git commit -m "Feat: Add main menu loop"
git push origin feature/quiz-play
```

Git commit, push

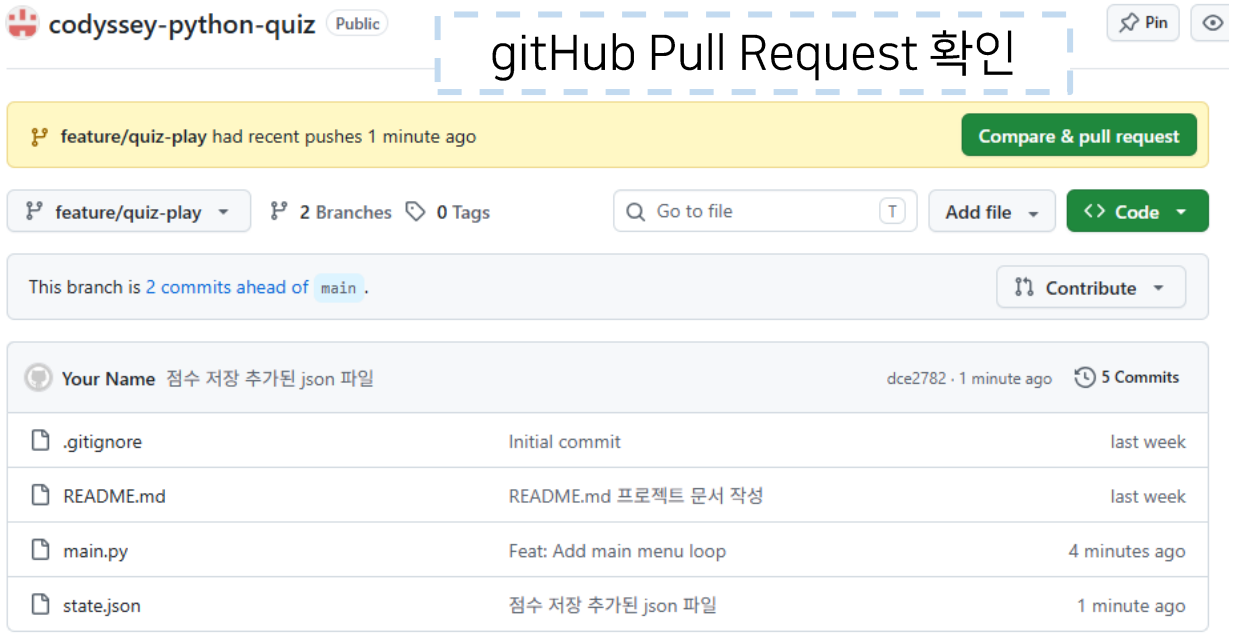
gitHub 반영 확인

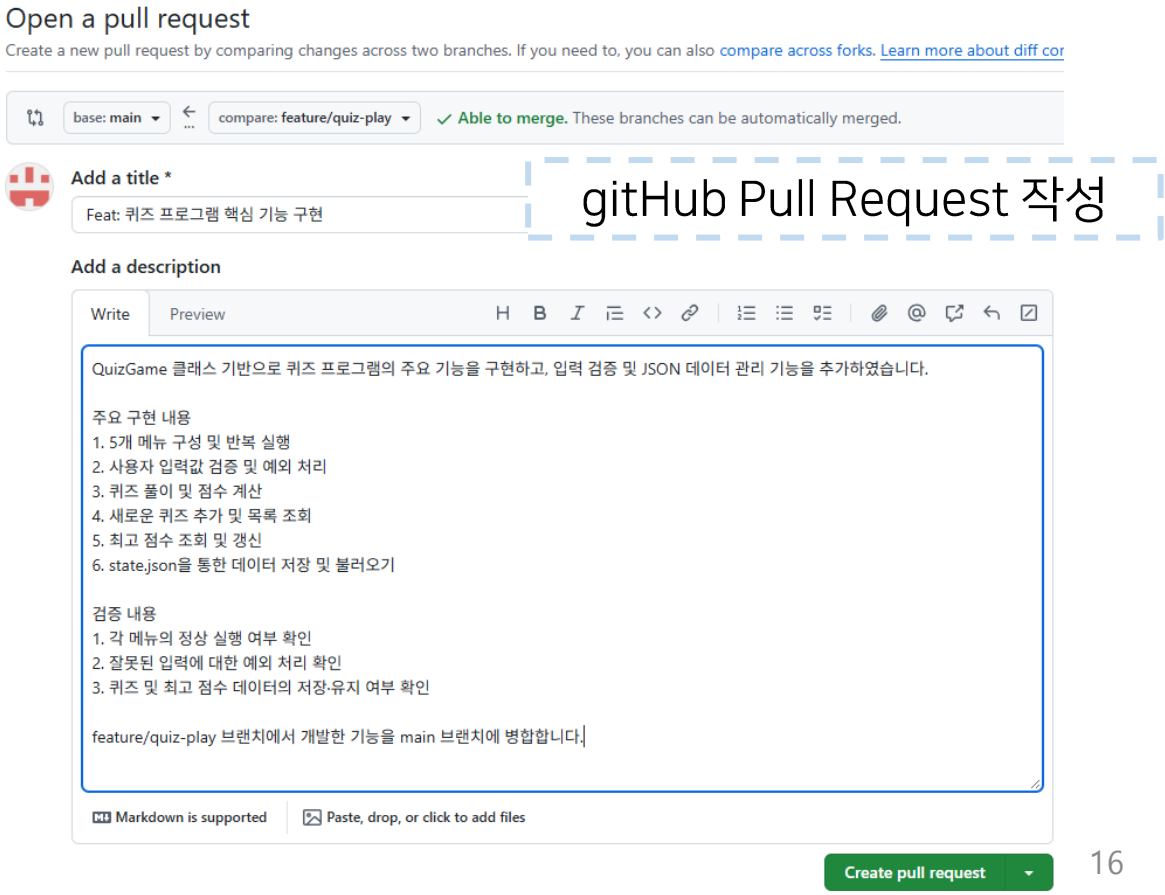


03 기능 요구사항별 구현 방법

■ 기능 구현 완료 후 브랜치 병합 및 프로그램 통합 - GitHub Pull Request 검토

- feature/quiz-play 브랜치에서 구현한 기능을 GitHub Pull Request로 검토
- Pull Request(PR)는 기능 개발 브랜치의 변경사항을 main 브랜치에 병합하기 위해 요청하는 기능으로, 변경 코드 검토와 의견 교환, 충돌 확인 등을 거쳐 안전하게 통합할 수 있도록 지원

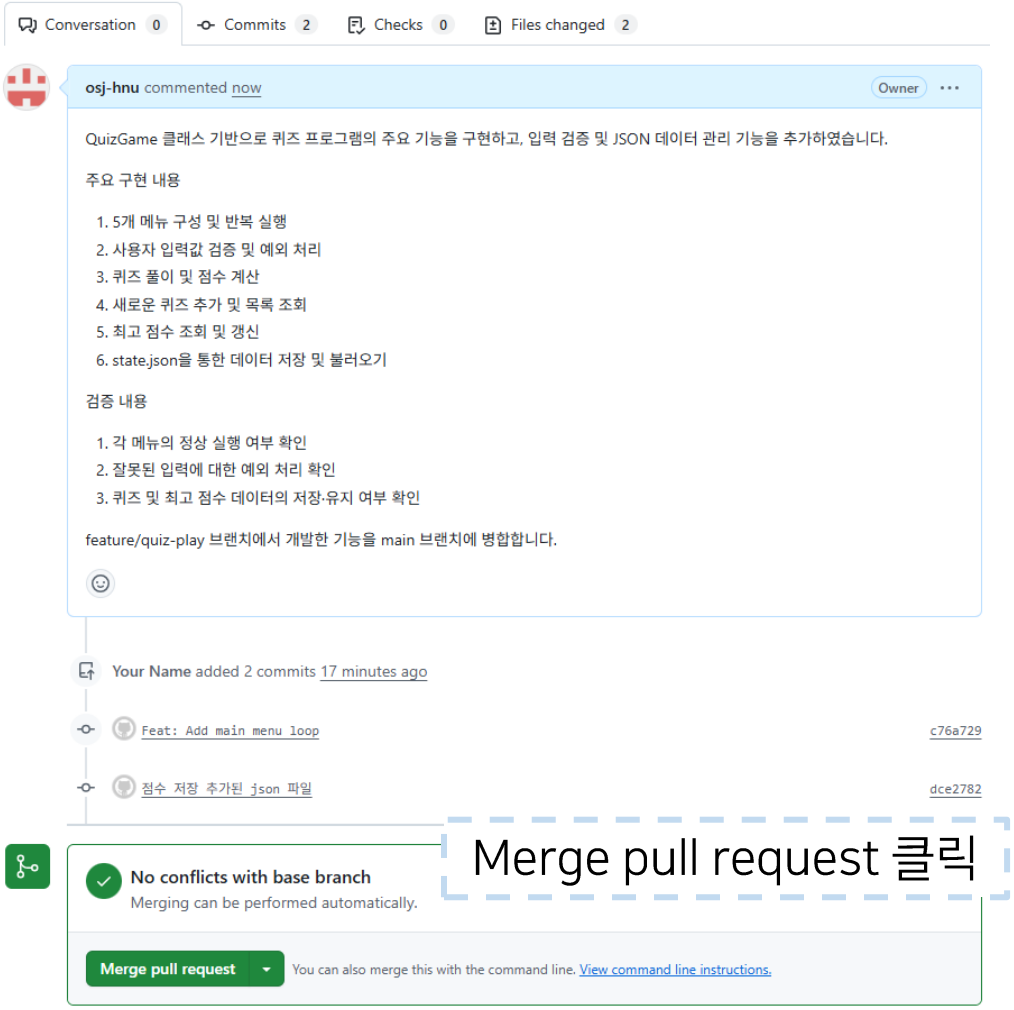




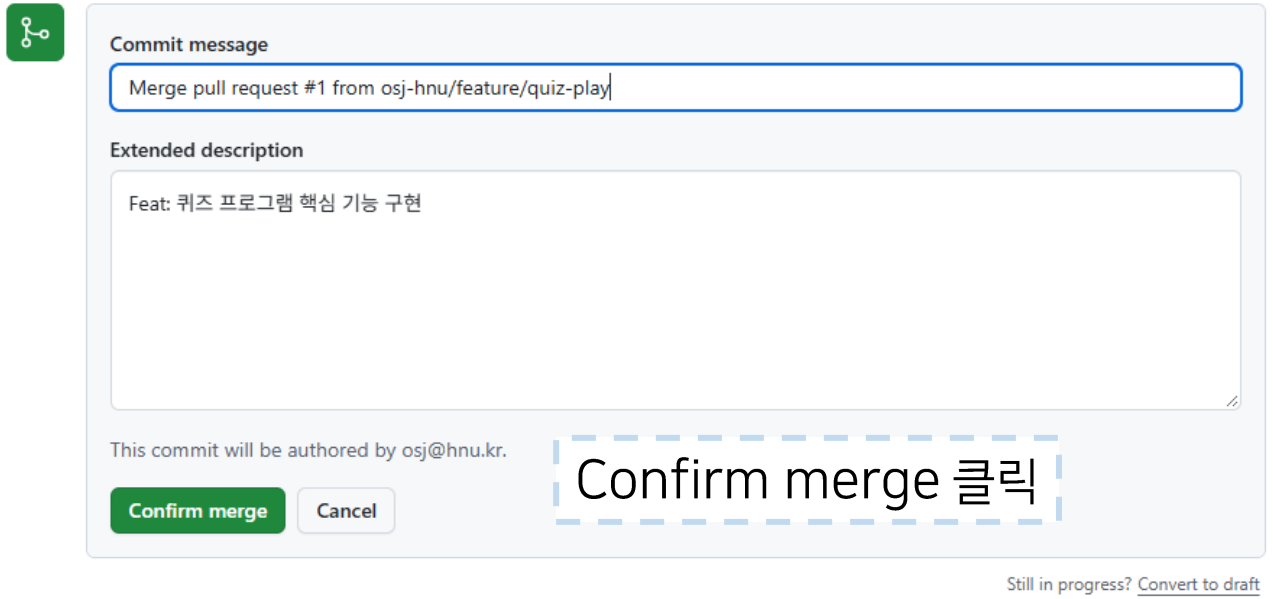
03 기능 요구사항별 구현 방법

■ 기능 구현 완료 후 브랜치 병합 및 프로그램 통합 – 브랜치를 main Merge

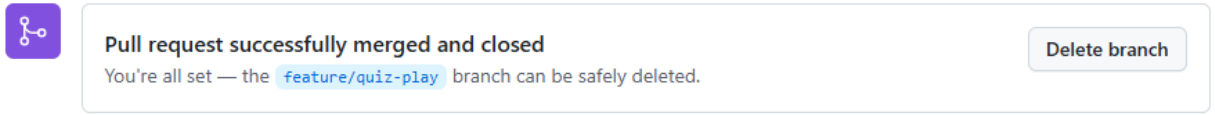
- main에 Merge한 후, 로컬 저장소를 동기화하여 전체 프로그램의 정상 실행 여부를 검증



Merge pull request 클릭



Confirm merge 클릭



feature/quiz-play의 변경사항이
GitHub의 main 브랜치에 반영 완료

03 기능 요구사항별 구현 방법

■ 기능 구현 완료 후 브랜치 병합 및 프로그램 통합 - VS Code에서 로컬 main 갱신

- GitHub Merge 및 로컬 main 브랜치 동기화 VS Code 터미널에서 실행

```
git switch main
git pull origin main
git status
python main.py
```

명령어	설명
git switch main	현재 작업 브랜치를 기본 브랜치인 main으로 전환
git pull origin main	GitHub에서 병합된 최신 main 브랜치의 변경사항을 로컬 main에 반영
git status	현재 브랜치와 작업 파일의 변경 상태를 확인하여 동기화 여부 검증
python main.py	병합된 퀴즈 프로그램을 실행하여 주요 기능의 정상 작동 여부 확인

- VS Code에서 코드 실행하여, 프로그램 기능 확인 후 README.md 파일 최종 수정 후 commit, push 진행

1. 기능 구현: 5개 메뉴, 퀴즈 풀이·추가, 목록 조회, 최고 점수 관리
2. 객체지향 구조: Quiz, QuizGame 클래스와 필수 메서드 구현
3. 데이터 관리: state.json 저장·불러오기 및 예외 처리
4. 문서화: README.md의 필수 내용과 실제 구현 내용 일치 여부

04 프로그램 검증 및 Git 버전 관리

■ 미션 필수 요구사항과 산출물 내용을 기반으로 프로그램 기능 검증

1. 기능 구현: 5개 메뉴, 퀴즈 풀이·추가, 목록 조회, 최고 점수 관리
2. 객체지향 구조: Quiz, QuizGame 클래스와 필수 메서드 구현
3. 데이터 관리: state.json 저장·불러오기 및 예외 처리
4. 문서화: README.md의 필수 내용과 실제 구현 내용 일치 여부
5. Git 이력: Commit 10회 이상, 브랜치 생성·Merge, Clone·Push·Pull 수행 여부

```
git log --oneline --graph --all –decorate
git reflog
```

```
PS D:\codyssey-python-quiz> git log --oneline --graph --all --decorate
*   fa81d9d (HEAD -> main, origin/main, origin/HEAD) Merge pull request #1 from osj-hnu/feature/quiz-play
|\
| * dce2782 (origin/feature/quiz-play, feature/quiz-play) 점수 저장 추가된 json 파일
| * c76a729 Feat: Add main menu loop
|/
* bdbe177 Add Quiz and QuizGame classes with initial data
* ede8b07 README.md 프로젝트 문서 작성
* bb9b916 Initial commit
```

```
PS D:\codyssey-python-quiz> git reflog
fa81d9d (HEAD -> main, origin/main, origin/HEAD) HEAD@{0}: pull origin main: Fast-forward
bdbe177 HEAD@{1}: checkout: moving from feature/quiz-play to main
dce2782 (origin/feature/quiz-play, feature/quiz-play) HEAD@{2}: commit: 점수 저장 추가된 json 파일
c76a729 HEAD@{3}: commit: Feat: Add main menu loop
bdbe177 HEAD@{4}: checkout: moving from main to feature/quiz-play
bdbe177 HEAD@{5}: checkout: moving from main to main
bdbe177 HEAD@{6}: commit: Add Quiz and QuizGame classes with initial data
ede8b07 HEAD@{7}: clone: from https://github.com/osj-hnu/codyssey-python-quiz.git
```

04 프로그램 검증 및 Git 버전 관리

■ 최종 산출물

- 퀴즈 게임 실행 파일(main.py), 퀴즈 데이터 저장 파일(state.json), 프로젝트 설명 문서(README.md)를 완성하고, GitHub 저장소에 최종 코드와 개발 이력을 반영

산출물	주요 내용
main.py	퀴즈 게임의 주요 기능과 Quiz·QuizGame 클래스를 구현한 Python 실행 파일
state.json	퀴즈 목록과 최고 점수를 JSON 형식으로 저장하는 데이터 파일
README.md	프로젝트 개요, 실행 방법, 주요 기능 및 파일 구조를 설명하는 문서
GitHub 저장소	최종 소스코드와 문서를 관리하고 Commit 10회 이상, 브랜치 생성·Merge 및 Clone·Pull 수행 이력을 확인할 수 있는 저장소

Thank you for your attention